Міністерство освіти і науки України
Донецький національний університет імені Василя Стуса Факультет інформаційних і прикладних технологій
Кафедра інформаційних технологій

ЗВІТ

з лабораторної роботи № 13

з дисципліни «Основи програмування»

на тему:

«Базові елементи бібліотеки tkinter»

Виконав: студент гр. Б25_д/F3
Кручківський Ю.О.
Перевірив: доц. Бабаков Р. М.

Вінниця – 2025

Зміст

Визначення завдання 3
Код до завдання 3
Код UI/gui/TkinterViewer.py 3
Код modules/metal_calc/models/materials.py 7
Код modules/metal_calc/models/products.py 7
Код modules/metal_calc/schemas/parameters.py 9
Код modules/metal_calc/commands.py 10
Код modules/metal_calc/service.py 11

Визначення завдання

Для завдання було обрано тему “Калькулятор для виробів з тонколистових металів”. Було реалізовано три команди по заданій специфіці

- Показати доступні тонколистові метали
- Розрахувати розгортку та вартість труби
- Розрахувати приблизну вартість коліна

В якості основи для програми було використано раніше розроблений програмний комплекс в якості індивідуального творчого завдання

Код до завдання

Код UI/gui/TkinterViewer.py

```
import tkinter as tk
from core.events import Event, EventType


class TkinterViewer:
    def __init__(self, context):
        self.context = context
        self.root = tk.Tk()
        self.root.title("Individual Creative Assignment")
        self.root.geometry("1000x600")

        self.root.protocol("WM_DELETE_WINDOW", self.on_closing)

        self.services = {"Вийти": ""}
        self.commands = {}

        self.sidebar = tk.Frame(self.root, width=300, bg="#ffffff",
relief="sunken", bd=1)
        self.sidebar.pack(side=tk.LEFT, fill=tk.Y, padx=5, pady=5)
        self.sidebar.pack_propagate(False)

        self.footer = tk.Frame(self.root, height=30, bg="#ffffff",
relief="sunken", bd=1)
        self.footer.pack(side=tk.BOTTOM, fill=tk.X, padx=5, pady=5)

        self.status = tk.Label(self.footer, text="Ready",
fg="black", bg="#ffffff", anchor="w")

        self.status.pack(fill=tk.X)

        self.main_area = tk.Frame(self.root, bg="white")
        self.main_area.pack(side=tk.RIGHT, fill=tk.BOTH,
expand=True, padx=5, pady=5)

        self.current_params_values = {}
        self.current_params_widgets = {}

    def on_closing(self):
        self.context.events.emit(Event(EventType.EXIT))

    def handle_event(self, event):
        self.status.config(text=event)
        if event.type == EventType.START:
            self.show_services_sidebar()

        elif event.type == EventType.SERVICE_AVAILABLE:
```

```
                service_name = event.data["service_name"]
                self.services[service_name] = event.data["service"]
                self.show_services_sidebar()

            elif event.type == EventType.COMMAND_AVAILABLE:
                command_name = event.data['name']
                self.commands[command_name] = {
                    "command_cls": event.data["command_cls"],
                    "service": event.data["service_name"]
                }

            elif event.type == EventType.SHOW_SERVICES_MENU:
                self.show_services_sidebar()

            elif event.type == EventType.SERVICE_SELECTED:
                self.show_commands_sidebar(event.data["service_name"])

            elif event.type == EventType.EXECUTE_COMMAND:
                self.show_params_form(event.data["service_name"],
event.data["command_name"])

            elif event.type == EventType.COMMAND_EXECUTED:
                self.show_result("Результат",
str(event.data["result"]))

            elif event.type == EventType.ERROR:
                self.show_result("Помилка",
str(event.data["error_message"]))

            elif event.type == EventType.EXIT:
                self.root.destroy()


        def clear_sidebar(self):
            for widget in self.sidebar.winfo_children():
                widget.destroy()

        def clear_main_area(self):
            for widget in self.main_area.winfo_children():
                widget.destroy()

        def show_services_sidebar(self):
            self.clear_sidebar()
            tk.Label(self.sidebar, text="Сервіси", font=("Arial", 14,
"bold"), bg="#f0f0f0").pack(pady=10)

            for name in self.services:
                if name == "Вийти":
                    tk.Button(self.sidebar, text=name, width=300,
command=self.on_closing, bg="#ff9999").pack(
                        side=tk.BOTTOM)
                else:
                    tk.Button(self.sidebar, text=name, width=300,
                              command=lambda n=name:
self.context.events.emit(
                                  Event(EventType.SERVICE_SELECTED,
service_name=n))).pack()

        def show_commands_sidebar(self, service_name):
            self.clear_sidebar()
```

```
            tk.Label(self.sidebar, text=f"Команди\n{service_name}",
font=("Arial", 12, "bold"), bg="#f0f0f0").pack(pady=10)

            commands_for_service = [k for k, v in self.commands.items()
if v["service"] == service_name]

            for cmd_name in commands_for_service:
                tk.Button(self.sidebar, text=cmd_name, width=300,
                          command=lambda c=cmd_name:
self.context.events.emit(
                              Event(EventType.EXECUTE_COMMAND,
command_name=c, service_name=service_name)
                          )).pack()

            tk.Button(self.sidebar, text="<- До сервісів", width=300,
command=self.show_services_sidebar,
                      bg="#cccccc").pack(
                side=tk.BOTTOM)

        def show_result(self, title, message, color="white"):
            self.clear_main_area()
            self.main_area.configure(bg=color)
            tk.Label(self.main_area, text=title, font=("Arial", 16,
"bold"), bg=color).pack(pady=20)
            tk.Label(self.main_area, text=message, font=("Arial", 12),
bg=color, justify=tk.LEFT, wraplength=600).pack(
                padx=20, pady=10, fill=tk.BOTH, expand=True)
            tk.Button(self.main_area, text="Очистити", width=20,
command=self.clear_main_area).pack(pady=20)

        def show_params_form(self, service_name, command_name):
            self.clear_main_area()
            self.main_area.configure(bg="white")
            service = self.services[service_name]
            command_cls = self.commands[command_name]["command_cls"]

            tk.Label(self.main_area, text=f"Команда: {command_name}",
font=("Arial", 14, "bold"), bg="white").pack(pady=10)
            if hasattr(command_cls, "description"):
                tk.Label(self.main_area, text=command_cls.description,
wraplength=600, bg="white").pack(pady=5)

            params = []
            if hasattr(command_cls, "get_params"):
                params = command_cls.get_params(service)
            elif hasattr(command_cls, "expected_params"):
                params = command_cls.expected_params

            if not params:
                self.execute_with_params(command_cls, service, [])
                return

            self.current_params_values = {}
            self.current_params_error_labels = {}

            form_frame = tk.Frame(self.main_area, bg="white")
            form_frame.pack(fill=tk.BOTH, expand=True, padx=20)

            for param in params:
                param_container = tk.Frame(form_frame, bg="white")
```

```
                param_container.pack(fill=tk.X, pady=2)

                param_frame = tk.Frame(param_container, bg="white")
                param_frame.pack(fill=tk.X)

                tk.Label(param_frame, text=f"{param.display_name}:",
width=20, anchor="w", bg="white").pack(side=tk.LEFT)
                tk.Label(param_frame, text=f"{param.description}:",
width=20, anchor="w", bg="white").pack(side=tk.RIGHT)

                if param.parse == bool:
                    var = tk.BooleanVar(value=False)
                    self.current_params_values[param.name] = var
                    tk.Checkbutton(param_frame, text=param.description,
variable=var, bg="white").pack(side=tk.LEFT)

                elif param.parse in (int, float):
                    if param.name == "diameter":
                        val_var = tk.DoubleVar(value=0.0) if
param.parse == float else tk.IntVar(value=0)
                        self.current_params_values[param.name] =
val_var
                        values = (100, 125, 150, 200, 250, 315)
                        for value in values:
                            tk.Radiobutton(param_frame, text=value,
value=value, variable=val_var).pack(side=tk.LEFT)
                    else:
                        val_var = tk.DoubleVar(value=0.0) if
param.parse == float else tk.IntVar(value=0)
                        self.current_params_values[param.name] =
val_var

                        tk.Button(param_frame, text="-100",
command=lambda v=val_var: v.set(v.get() - 100)).pack(side=tk.LEFT)
                        tk.Button(param_frame, text="-10",
command=lambda v=val_var: v.set(v.get() - 10)).pack(side=tk.LEFT)
                        tk.Button(param_frame, text="-1",
command=lambda v=val_var: v.set(v.get() - 1)).pack(side=tk.LEFT)

                        label = tk.Label(param_frame,
textvariable=val_var, width=10, relief="sunken")
                        label.pack(side=tk.LEFT, padx=5)

                        tk.Button(param_frame, text="+1",
command=lambda v=val_var: v.set(v.get() + 1)).pack(side=tk.LEFT)
                        tk.Button(param_frame, text="+10",
command=lambda v=val_var: v.set(v.get() + 10)).pack(side=tk.LEFT)
                        tk.Button(param_frame, text="+100",
command=lambda v=val_var: v.set(v.get() + 100)).pack(side=tk.LEFT)

                error_label = tk.Label(param_container, text="",
fg="red", bg="white", font=("Arial", 8))
                error_label.pack(anchor="w", padx=(145, 0))
                self.current_params_error_labels[param.name] =
error_label

            tk.Button(self.main_area, text="Виконати", bg="#99ff99",
height=2, width=30,
                      command=lambda:
self.collect_and_execute(command_cls, service, params)).pack(pady=20)
```

```
            tk.Button(self.main_area, text="Скасувати",
command=self.clear_main_area).pack()

        def collect_and_execute(self, command_cls, service,
params_meta):
            args = []
            is_valid = True

            for label in self.current_params_error_labels.values():
                label.config(text="")

            for p in params_meta:
                try:
                    val = self.current_params_values[p.name].get()
                    converted_val = p.convert(str(val))
                    args.append(converted_val)
                except (ValueError, tk.TclError) as e:
                    is_valid = False
                    error_msg = str(e)
                    if isinstance(e, tk.TclError):
                        error_msg = "Невірний формат числа"

self.current_params_error_labels[p.name].config(text=error_msg)

            if is_valid:
                self.execute_with_params(command_cls, service, args)

        def execute_with_params(self, command_cls, service, args):
            cmd_instance = command_cls(service, *args)
            self.context.events.emit(Event(EventType.RUN_COMMAND,
command=cmd_instance))

        def update_gui(self):
            try:
                self.root.update()
            except tk.TclError:
                pass
```

Код modules/metal_calc/models/materials.py

```
from dataclasses import dataclass


@dataclass
class MetalMaterial:
    id: int
    name: str
    price_per_m2: float


THIN_METALS = [
    MetalMaterial(0, "Сталь (S235) 1мм", 450.0),
    MetalMaterial(1, "Сталь (S235) 2мм", 850.0),
    MetalMaterial(2, "Нержавіюча сталь 1мм", 1200.0),
    MetalMaterial(3, "Алюміній 1.5мм", 950.0)
]
```

Код modules/metal_calc/models/products.py

```
import math
from dataclasses import dataclass
from modules.metal_calc.models.materials import MetalMaterial
```

```
@dataclass
class Product:
    diameter_mm: float
    material: MetalMaterial
    has_salary: bool

    def get_area_m2(self) -> float:
        raise NotImplementedError

    def get_salary(self) -> float:
        raise NotImplementedError

    def get_cost(self) -> float:
        return (self.get_area_m2() * self.material.price_per_m2) +
(self.get_salary() if self.has_salary else 0)


@dataclass
class Pipe(Product):
    length_mm: float

    def get_salary(self) -> float:
        return self.get_area_m2() * 50

    def get_width_mm(self) -> float:
        return self.diameter_mm * math.pi

    def get_area_m2(self) -> float:
        width_m = self.get_width_mm() / 1000
        length_m = self.length_mm / 1000
        return width_m * length_m

    def __str__(self):
        return (f"Виріб: Труба (Діаметр: {self.diameter_mm} мм,
Довжина: {self.length_mm} мм)\n"
                f"Розгортка: {self.get_width_mm():.2f} x
{self.length_mm} мм\n"
                f"Матеріал: {self.material.name}\n"
                f"Площа: {self.get_area_m2():.3f} м2\n"
                f"Вартість: {self.get_cost():.2f} грн")


@dataclass
class Elbow(Product):
    angle_deg: float
    segments: int

    def get_salary(self) -> float:
        return self.get_area_m2() * 50

    def get_radius_bend(self) -> float:
        return self.diameter_mm * 1.5

    def get_arc_length_mm(self) -> float:
        return (self.get_radius_bend() * 2 * math.pi) *
(self.angle_deg / 360)

    def get_area_m2(self) -> float:
```

```
            width_m = (self.diameter_mm * math.pi) / 1000
            arc_length_m = self.get_arc_length_mm() / 1000
            return width_m * arc_length_m

        def __str__(self):
            return (f"Виріб: Коліно (Діаметр: {self.diameter_mm} мм,
Кут: {self.angle_deg}°, Сегментів: {self.segments})\n"
                    f"Матеріал: {self.material.name}\n"
                    f"Приблизна площа: {self.get_area_m2():.3f} м2\n"
                    f"Приблизна вартість: {self.get_cost():.2f} грн\n"
                    f"(Розрахунок базується на середній довжині дуги
при R=1.5D)")
```

Код modules/metal_calc/schemas/parameters.py

```
    from core.parameter import ParameterSchema, IndexParameter,
Parameter, BoolParameter

    MATERIAL_IDX = ParameterSchema(
        name="material_index",
        display_name="Матеріал",
        description="Вибір індексу матеріалу",
        parse_type=int,
        parameter_cls=IndexParameter
    )

    DIAMETER = ParameterSchema(
        name="diameter",
        display_name="Діаметр",
        description="Діаметр в мм",
        parse_type=float
    )

    LENGTH = ParameterSchema(
        name="length",
        display_name="Довжина",
        description="Довжина в мм",
        parse_type=float
    )

    ANGLE = ParameterSchema(
        name="angle",
        display_name="Кут",
        description="Кут в градусах",
        parse_type=float
    )

    SEGMENTS = ParameterSchema(
        name="segments",
        display_name="Сегменти",
        description="Кількість сегментів",
        parse_type=int
    )

    HAS_SALARY = ParameterSchema(
        name="has_salary",
        display_name="Заробітня плата",
        description="Чи включається заробітня плата",
        parse_type=bool,
        parameter_cls=BoolParameter
    )
```

Код modules/metal_calc/commands.py

```
from core.command import Command
from modules.metal_calc.schemas.parameters import *
from abc import ABC


class CalculateMetalCommand(Command, ABC):
    @classmethod
    def get_params(cls, service):
        materials_count = service.get_materials_count()
        return [
            HAS_SALARY.build(),
            MATERIAL_IDX.with_range(1,
materials_count).build(custom_desc=f"від 1 до {materials_count}")
        ]

    def __init__(self, service, has_salary, material_index):
        self.service = service
        self.has_salary = has_salary
        self.material_index = material_index


class CalculateMetalRoundCommand(CalculateMetalCommand, ABC):
    @classmethod
    def get_params(cls, service):
        return super().get_params(service) + [
            DIAMETER.with_range(min_val=1).build()
        ]

    def __init__(self, service, has_salary, material_index,
diameter):
        super().__init__(service, has_salary, material_index)
        self.diameter = diameter


class GetMaterialsCommand(Command):
    name = "Довідник матеріалів"
    description = "Показати доступні тонколистові метали"

    def __init__(self, service):
        self.service = service

    def _execute(self, context):
        return self.service.get_materials()


class CalculatePipeCommand(CalculateMetalRoundCommand):
    name = "Розрахувати трубу"
    description = "Розрахувати розгортку та вартість труби"

    @classmethod
    def get_params(cls, service):
        return super().get_params(service) + [
            LENGTH.with_range(min_val=1).build()
        ]

    def __init__(self, service, has_salary, material_index,
diameter, length):
```

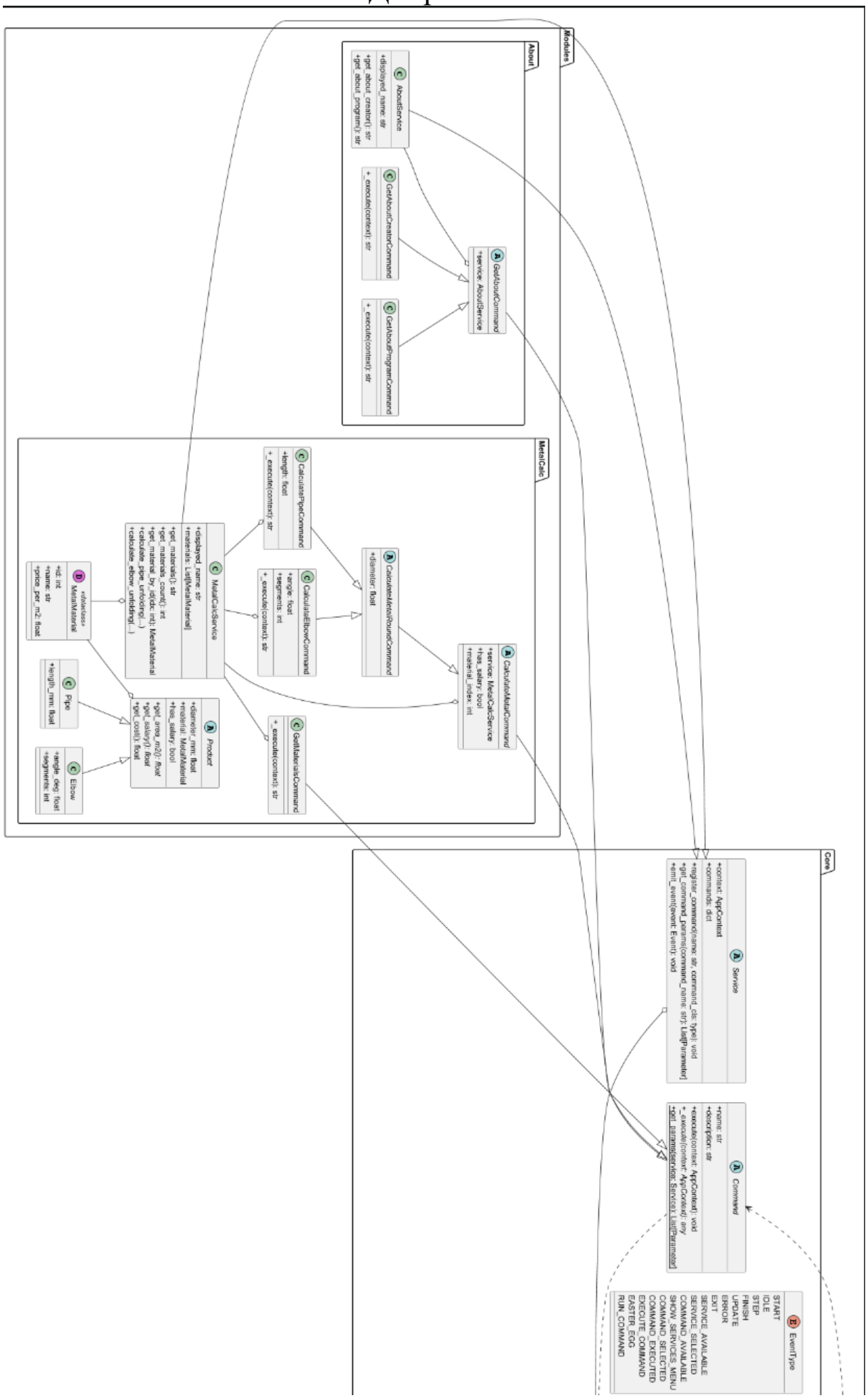
```
            super().__init__(service, has_salary, material_index,
diameter)
            self.length = length

        def _execute(self, context):
            return self.service.calculate_pipe_unfolding(self.diameter,
self.length, self.material_index, self.has_salary)


    class CalculateElbowCommand(CalculateMetalRoundCommand):
        name = "Розрахувати коліно"
        description = "Розрахувати приблизну вартість коліна"

        @classmethod
        def get_params(cls, service):
            return super().get_params(service) + [
                ANGLE.with_range(0, 360).build(),
                SEGMENTS.with_range(min_val=1).build()
            ]

        def __init__(self, service, has_salary, material_index,
diameter, angle, segments):
            super().__init__(service, has_salary, material_index,
diameter)
            self.angle = angle
            self.segments = segments

        def _execute(self, context):
            return
self.service.calculate_elbow_unfolding(self.diameter, self.angle,
self.segments, self.material_index, self.has_salary)
```

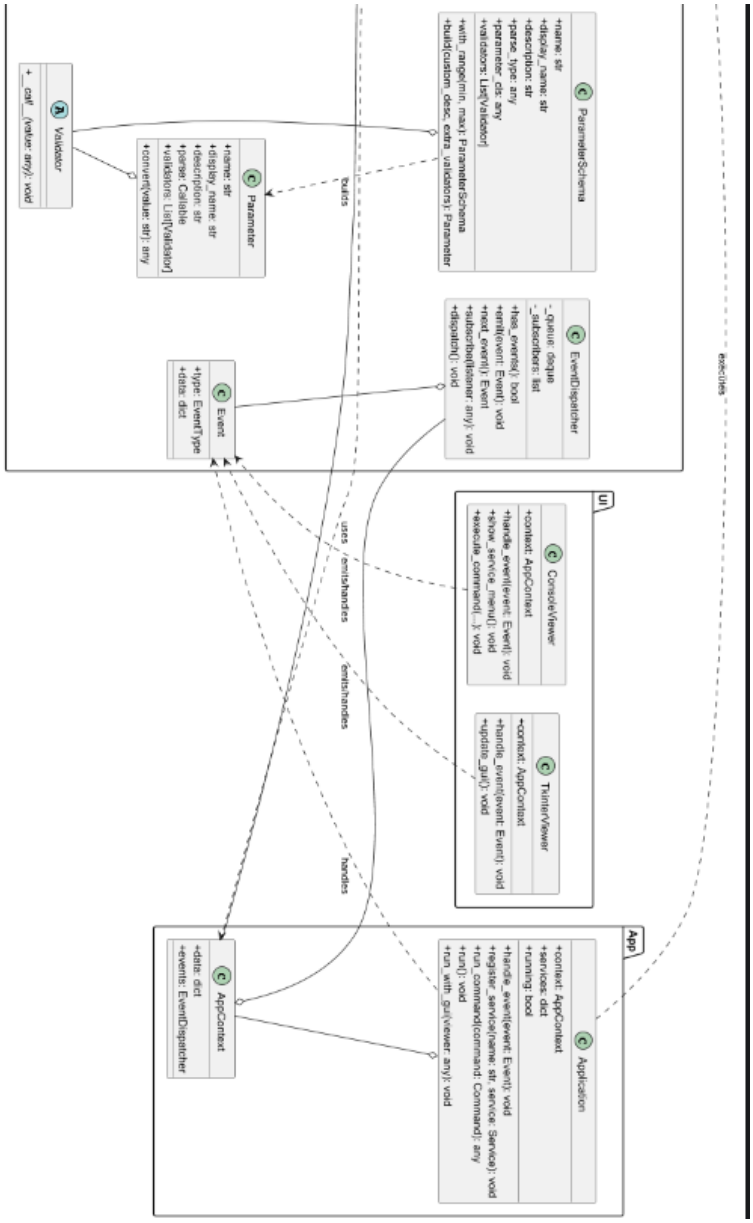
Код modules/metal_calc/service.py

```
    from core.service import Service
    from modules.metal_calc.commands import *
    from modules.metal_calc.models.materials import THIN_METALS
    from modules.metal_calc.models.products import Pipe, Elbow


    class MetalCalcService(Service):
        displayed_name = "Калькулятор тонколистового металу"

        def __init__(self, context):
            super().__init__(context)
            self.materials = THIN_METALS

            self.register_command(GetMaterialsCommand.name,
GetMaterialsCommand)
            self.register_command(CalculatePipeCommand.name,
CalculatePipeCommand)
            self.register_command(CalculateElbowCommand.name,
CalculateElbowCommand)

        def get_materials(self):
            result = "Доступні тонколистові метали (ціна за м2):\n"
            for material in sorted(self.materials, key=lambda x: x.id):
                result += f"{material.id + 1}. {material.name}:
{material.price_per_m2} грн/м2\n"
            return result.strip()
```

```
        def get_materials_count(self):
            return len(self.materials)

        def get_material_by_id(self, idx):
            for material in self.materials:
                if material.id == idx:
                    return material
            else:
                raise IndexError(f"Material with {idx} not defined")

        def calculate_pipe_unfolding(self, diameter_mm: float,
length_mm: float, material_index: int, has_salary: bool):
            material = self.get_material_by_id(material_index)
            pipe = Pipe(diameter_mm=diameter_mm, material=material,
length_mm=length_mm, has_salary=has_salary)
            return str(pipe)

        def calculate_elbow_unfolding(self, diameter_mm: float,
angle_deg: float, segments: int, material_index: int, has_salary: bool):
            material = self.get_material_by_id(material_index)
            elbow = Elbow(diameter_mm=diameter_mm, material=material,
angle_deg=angle_deg, segments=segments, has_salary=has_salary)
            return str(elbow)
```

Діаграма


Modules
About
AboutService
+displayed_name: str
+get_about_creator(): str
+get_about_program(): str
GetAboutCommand
+service: AboutService
GetAboutCreatorCommand
+_execute(context): str
GetAboutProgramCommand
+_execute(context): str
MetalCalc
CalculateMetalCommand
+service: MetalCalcService
+has_salary: bool
+material_index: int
CalculateMetalRoundCommand
+diameter: float
CalculateElbowCommand
+angle: float
+segments: int
+_execute(context): str
CalculatePipeCommand
+length: float
+_execute(context): str
GetMaterialsCommand
+_execute(context): str
MetalCalcService
+displayed_name: str
+materials: List[MetalMaterial]
+get_materials(): str
+get_materials_count(): int
+get_material_by_id(idx: int): MetalMaterial
+calculate_pipe_unfolding(...)
+calculate_elbow_unfolding(...)
Product
+diameter_mm: float
+material: MetalMaterial
+has_salary: bool
+get_area_m2(): float
+get_salary(): float
+get_cost(): float
«dataclass»
MetalMaterial
+id: int
+name: str
+price_per_m2: float
Pipe
+length_mm: float
Elbow
+angle_deg: float
+segments: int
Core
Service
+context: AppContext
+commands: dict
+register_command(name: str, command_cls: type): void
+get_command_params(command_name: str): List[Parameter]
+emit_event(event: Event): void
Command
+name: str
+description: str
+execute(context: AppContext): void
+_execute(context: AppContext): any
+get_params(service: Service): List[Parameter]
EventType
START
IDLE
STEP
FINISH
UPDATE
ERROR
EXIT
SERVICE_AVAILABLE
SERVICE_SELECTED
COMMAND_AVAILABLE
SHOW_SERVICES_MENU
COMMAND_SELECTED
COMMAND_EXECUTED
EXECUTE_COMMAND
EASTER_EGG
RUN_COMMAND


ParameterSchema
+name: str
+display_name: str
+description: str
+parse_type: any
+parameter_cls: any
+validators: List[Validator]
+with_range(min, max): ParameterSchema
+build(custom_desc, extra_validators): Parameter
EventDispatcher
-_queue: deque
-_subscribers: list
+has_events(): bool
+emit(event: Event): void
+next_event(): Event
+subscribe(listener: any): void
+dispatch(): void
builds
Parameter
+name: str
+display_name: str
+description: str
+parser: Callable
+validators: List[Validator]
+convert(value: str): any
Event
+type: EventType
+data: dict
Validator
+__call__(value: any): void
uses
emits/handles
emits/handles
handles
executes
ui
ConsoleViewer
+context: AppContext
+handle_event(event: Event): void
+show_service_menu(): void
+execute_command(...): void
TkinterViewer
+context: AppContext
+handle_event(event: Event): void
+update_gui(): void
App
Application
+context: AppContext
+services: dict
+running: bool
+handle_event(event: Event): void
+register_service(name: str, service: Service): void
+run_command(command: Command): any
+run(): void
+run_with_gui(viewer: any): void
AppContext
+data: dict
+events: EventDispatcher

Приклад використання модуля

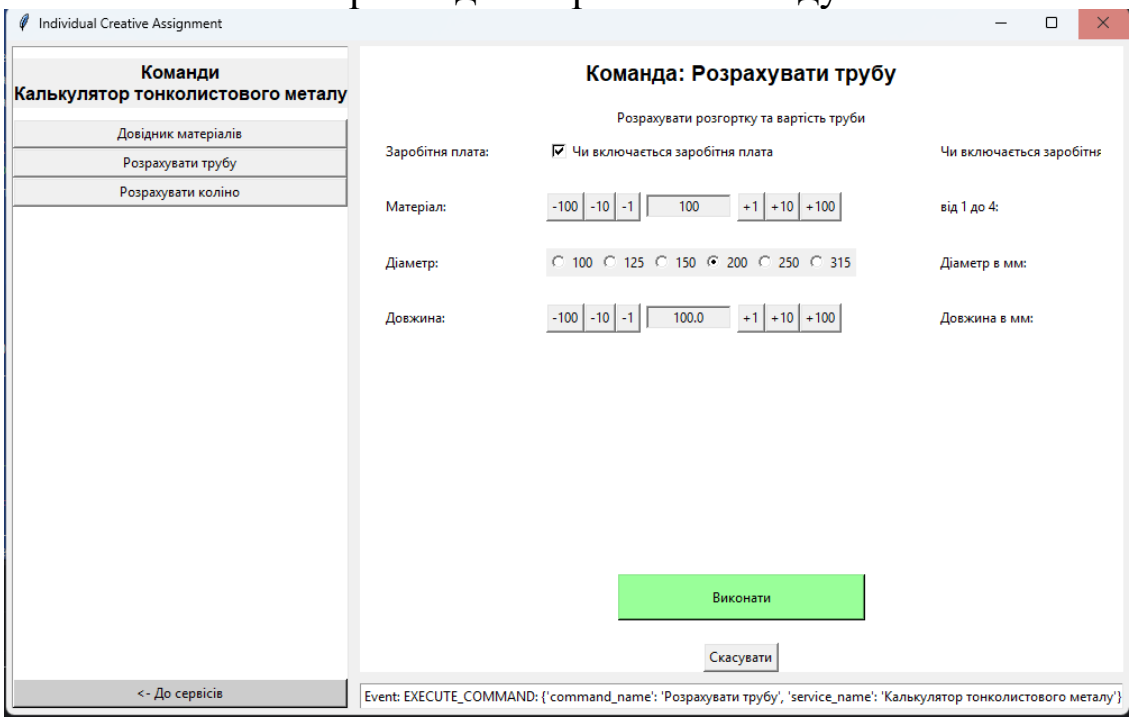


Рисунок 1 - приклад форми команди

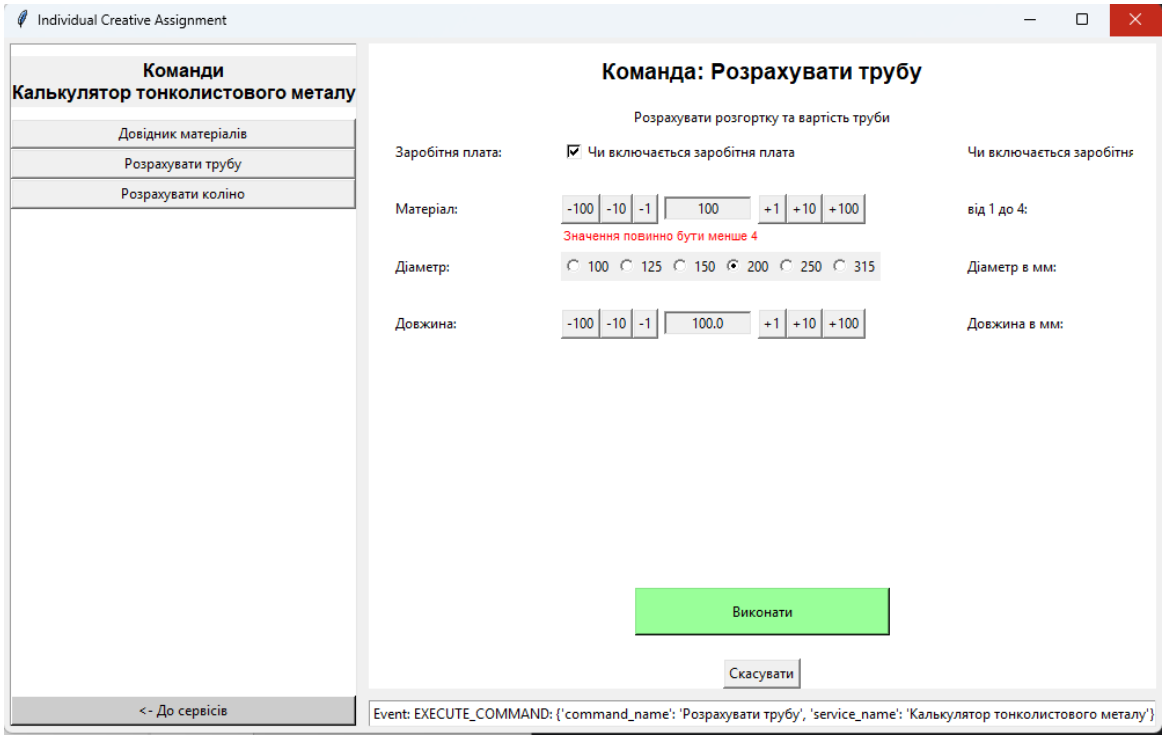


Рисунок 2 - приклад відображення не валідних параметрів

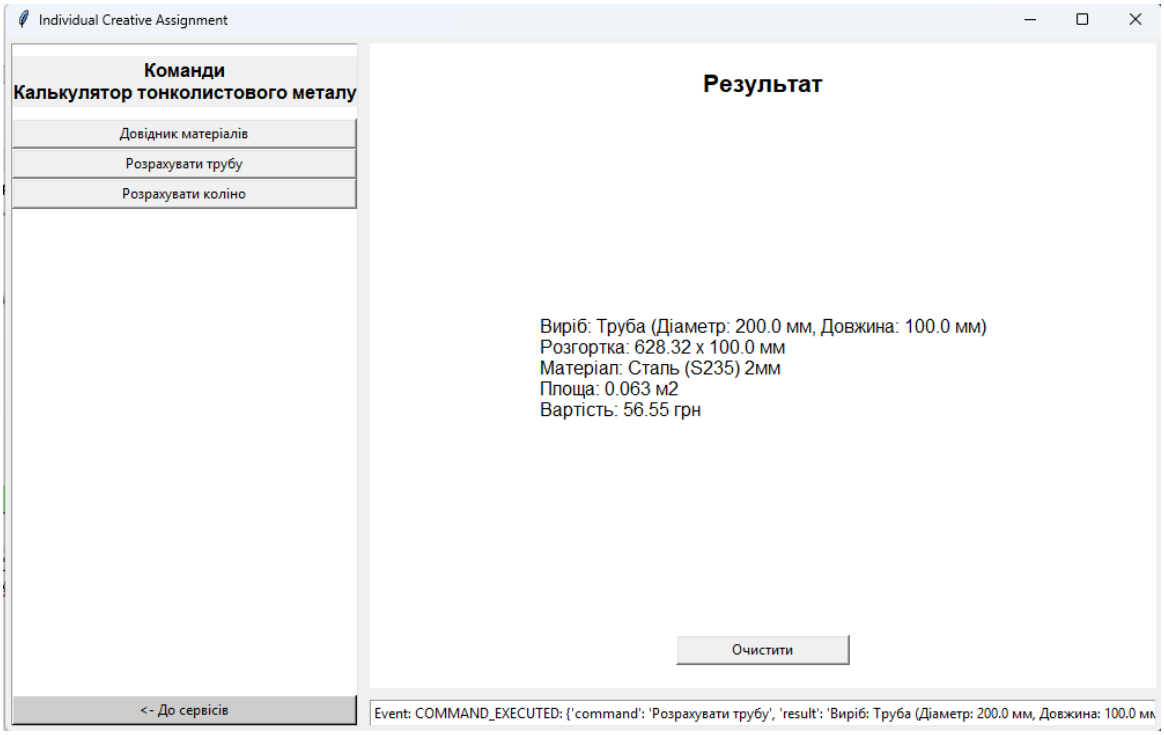


Рисунок 3 - приклад коректного виводу

Висновки

В ході виконання лабораторної було допрацьовано програмний комплекс та розроблено модуль для лабораторної роботи

Джерела

- Посилання на репозиторій https://github.com/Figeiron/IndividualCreativeAssignment/releases/tag/LabWork_12-v0.0.1